SIMATS ENGINEERING

Department of Information Technology

CSA37 – Software Testing

ASSIGNMENT 1 – CO1

Evaluate the Quality Process and Defect Profile of a Hospital Appointment Application

CO1: Testing viewed as an engineering activity, the role of process in software quality, testing as a process, basic definitions, principles of software testing, the role of the tester, origins of defects, defect classes, defect repository and defect examples.

Student Name	**Peta Harsha Vardhan**	**Register Number**	**192311039**
Department / Section	**CSE department**	**Academic Year / Semester**	**4th year /7th semister**
Faculty Name	**Dr.R Kesavan**	**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission	**4/11/2026**	**Date of Evaluation**	

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max	Marks Awarded	Remarks
1	Evaluate Testing as an Engineering Activity	5	**5**	All four engineering features (planning, measurement, standards, repeatability) judged against case evidence and mapped to incidents; engineered approach clearly stated.
2	Evaluate the Role of Process in Software Quality	2	**2**	Clear process-over-people position; Table 2 tests each incident; 38/100 coding-defect argument and TMM Level 1 placement well justified.
3	Evaluate Testing as a Process	6	**6**	Full test process, V&V per phase, detection phase per incident; cost ₹26,10,500 computed correctly; defect-flow model and limits explicitly stated.
4	Apply and Critique Basic Definitions	4	**4**	Definitions applied to MediQueue; error/fault/failure for I3, I5, I6 correct; I8 linked to a wrong oracle; glossary contents justified.
5	Evaluate the Application of Testing Principles	3	**3**	Six principles each tied to an incident or decision; "testing shows presence, not absence" used well; strongest principle defended with cost data.

6	Evaluate the Tester's Role	3	**3**	Clear position; tester role per phase, pros/cons of self testing, 7/2/1 catch table and healthcare-specific tester qualities.
7	Trace the Origins of Defects	4	**4**	Origin and evidence for all ten incidents with second-origin reasoning; pattern (50% requirements/design) matches the defect data.

8	Evaluate Defect Classes and Severity	4	**4**	Classification scheme with sub-classes; class, severity and priority for all ten; I7 vs I1 contrast explains severity ≠ priority.
9	Design and Appraise a Defect Repository	3	**3**	Field groups justified individually; analysis/improvement/prevention uses and e-mail vs repository comparison are convincing.
10	Evaluate Defect Examples and Prevention Strategies	4	**3.5**	Three classes, root causes and prevention strategies correct. Cost model rests on assumed ₹32,000 escape cost; overlapping savings noted but not quantified.
11	Recommend and Defend a Quality Improvement Plan	4	**3.5**	Prioritised, practical plan with 3.2% effort check. Expected effect depends on assumed 60%/35%/60% efficiencies that are not validated.
12A	Re-evaluate: Impact Analysis	2	**2**	Change table and affected assumptions clear; effect of two week releases and shared records on severity well analysed.
12B	Re-evaluate: Evaluation of the Original Plan	2	**2**	Capacity check (184 vs 140 hours, 31% over) supports the conclusion; new external-system defect origins and repository gaps identified.
12C	Re-evaluate: Adaptation or Redesign	2	**2**	Clear MODIFY decision; revised two-week cycle, tester role, principles and external-defect classification (Table 26) shown.
12D	Re-evaluate: Evidence Based Defence	2	**2**	Decision defended with defect, cost, capacity and patient safety evidence; accepted risks and review trigger stated.
TOTAL		**50**	**49**	**Percentage: 98%**

OVERALL REMARKS

Excellent, well-evidenced submission. Every task is answered with case data, the cost and defect flow calculations are correct and internally consistent (₹26,10,500; 60 production escapes; 9.6% early detection), and the Day-4 redesign follows logically from a capacity calculation. Minor weaknesses: the savings in Tasks 10–11 depend on assumed review/test efficiencies and cost figures that are stated but not validated, and the solution repeats each task's bullet list, which adds length without adding marks.

Student Signature	Faculty Signature	Program Director
		

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = present ✗ = missing / not satisfied P = partly present. Based on the "Evidence Expected" of each task.

Task	Evidence expected	Status	Note
1	A well-argued assessment of MediQueue's testing practice against the features of an engineering activity, backed by evidence from the case.	✓	Tables 1–2 and incident mapping
2	A well-justified position on how process affects quality, backed by data from the case.	✓	TMM placement justified
3	A process-oriented assessment of testing in MediQueue, backed by cost calculations from the defect data and a reasoned estimate of savings.	✓	Calculations shown; Figure 1 present
4	Accurate application of basic testing definitions to the MediQueue incidents and a justified judgement on how terminology affects quality.	✓	I3, I5, I6 and I8 covered
5	A justified assessment of the developers' assertion using testing principles tied to evidence from the case.	✓	Six principles linked to evidence
6	A defended judgement on how valuable an independent tester would be for MediQueue, tied to specific incidents.	✓	Position defended with catch table
7	A justified origin for every incident and an interpretation of the overall pattern of defect origins.	✓	All ten incidents traced
8	A justified classification of all ten incidents by class, severity and priority, together with a recommendation on which class to address first.	✓	Severity vs priority contrast given

9	A defect repository design for MediQueue with a justification for every field and an assessment of its benefits.	✓	Each field group justified
10	A root-cause analysis of three defects with justified prevention strategies that take cost into account.	**P**	Savings rest on assumed costs
11	A technically justified quality improvement plan backed by the evidence built up in earlier tasks.	**P**	Efficiencies assumed, not validated
12A–D	Impact analysis, evaluation of the original solution, adaptation or redesign, and evidence-based defence	✓	All four parts complete

DEFECT ANALYSIS EVIDENCE CHAIN (Tasks 3, 7–10)

Evidence item	**Status**	**Observation**
Defect data calculations (cost per phase, totals)	✓	Cost per phase and totals correct
Phase-containment / detection figures	✓	Table 7 flow; 40% detection before release

Origin assigned to every incident (I1– I10)	✓	I1–I10 with evidence and second origin
Class, severity and priority justified	✓	Table 17 with reasons
Defect repository fields with reasons	✓	Table 18 with reasons
Root cause → prevention strategy link	✓	Table 20
Cost–benefit of prevention strategies	**P**	Ratios 12.6 and 8.8 depend on assumptions
Interpretation of the defect pattern	✓	Pareto and investment conclusion
Figures and conclusions consistent	✓	Figures consistent across Tasks 1–12

SUBMISSION CHECKLIST (not separately marked)

Requirement	Status
All 12 tasks (including 12A–12D) answered	☒Yes ☐Partly ☐No
Defect cost calculations shown step by step	☒Yes ☐Partly ☐No
All ten incidents classified (origin, class, severity, priority)	☒Yes ☐Partly ☐No
Error, fault and failure separated for I3, I5 and I6	☒Yes ☐Partly ☐No
Testing principles linked to evidence from the case	☒Yes ☐Partly ☐No
Repository design justified field by field	☒Yes ☐Partly ☐No
Tables, figures and diagrams included and readable	☒Yes ☐Partly ☐No
Cross-references and table/figure numbering correct	☒Yes ☐Partly ☐No
Work consistent throughout	☒Yes ☐Partly ☐No
No unfilled template placeholders	☒Yes ☐Partly ☐No
Viva recommended	☐Yes ☒No

Software Testing (CSA37)

ASSIGNMENT 1

CO1: Testing viewed as an engineering activity, the role of process in software quality, testing as a process, basic definitions, principles of software testing, the role of the tester, origins of defects, defect classes, defect repository and defect examples. **Assignment Title: Evaluate the Quality Process and Defect Profile of a Hospital Appointment Application (Total Marks - 50)**

Duration:1week

Scenario — Hospital Outpatient Appointment Application (MediQueue)

MediQueue is a mobile and web application introduced by a multi-speciality hospital. Through it, patients can book outpatient appointments, pay consultation fees, obtain token numbers and view their prescriptions. Roughly 6,000 patients use the application each day.

A team of five developers built the application over eight months, with no dedicated tester. The developers "tested" their own code shortly before release, and the release was signed off because the demonstration to hospital management showed no errors.

In the three months following release, the hospital received a large number of complaints. As the newly appointed Quality Engineer, management has asked you to find out why so many defects reached patients and to recommend how quality can be built into the process.

Incidents logged after release:

ID	Incident
I1	I1 When two patients book at exactly the same second, both are issued the same token number for the same doctor.
I2	The system accepts Sunday appointments even though the outpatient department does not operate on Sundays; this rule was never captured in the requirements.
I3	The booking page displays a consultation fee of ₹500, yet ₹550 is collected; the GST rule was misinterpreted.
I4	The application crashes whenever a patient name is longer than 40 characters.
I5	Patients who reach the age of 60 on the appointment day do not receive senior-citizen priority.
I6	When two doctors save at the same moment, the prescription PDF displays the details of the previous patient.

I7	**The user manual instructs patients to cancel via "My Visits", but no such menu exists.**
I8	**A refund test case was recorded as "Pass" because the tester checked the output against an incorrect expected value.**
I9	**SMS reminders stop being sent when the hospital's SMS provider changes its API key.**
I10	**On the hospital's Wi-Fi network, the doctor's schedule screen needs 25 seconds to load.**

Defect data from the most recent release (100 defects):

Phase	Defects introduced	Defects detected	Cost to fix each defect (₹)	Remarks
Requirements	**30**	**2**	**500**	**No formal review**
Design	**22**	**3**	**1,500**	**No formal review**
Coding	**38**	**15**	**3,000**	**Developer self-check only**
Testing	**5**	**20**	**8,000**	**Demo based testing**
Documentation	**5**	**0**	**–**	**–**
Production (after release)	**–**	**60**	**40,000**	**Reported by patients**

Design Challenge:

The hospital intends to release a new version every two weeks and to link MediQueue with the national digital health ID system.

Reassess the quality process you recommended and decide whether it should be retained, modified or redesigned.

TASK 1 – Evaluate Testing as an Engineering Activity 5 Marks

CO1 | Bloom's Level: K5 – Evaluate

Examine whether the way MediQueue was tested qualifies as an engineering activity.

Did the MediQueue team deliberately engineer quality into the product, or did they simply hope it would be there?

Discuss:

- **The features of testing as an engineering activity (planning, measurement, standards, repeatability)**
- **Which of these features were absent in the MediQueue project, supported by evidence from the case**
- **How the absence of each feature led to the incidents that were reported**
- **What an engineered approach to testing would have looked like for this project**

You are required to assess the project against engineering practice rather than just define the term.

Evidence Expected

A well-argued assessment of MediQueue's testing practice against the features of an engineering activity, backed by evidence from the case.

TASK 2 – Evaluate the Role of Process in Software Quality 2 Marks

CO1 | Bloom's Level: K5 – Evaluate

Hospital management is convinced that employing "better developers" will fix the quality problem.

Does quality come mainly from good people or from a good

process? Discuss:

- **How a defined process contributes to achieving software quality**
- **Whether better developers on their own would have prevented the incidents**
- **Where a process maturity view (for example, TMM levels) would position the current team**

You must defend your position using evidence from the

case. Evidence Expected

A well-justified position on how process affects quality, backed by data from the case.

TASK 3 – Evaluate Testing as a Process 6 Marks

CO1 | Bloom's Level: K5 – Evaluate

In MediQueue, testing was carried out as one activity immediately before release.

How would the defect data have differed if testing had been treated as a process spanning the entire life cycle?

Discuss:

- **Testing as a process made up of planning, design, execution, evaluation and reporting activities**
- **How verification and validation activities relate to each development phase**
- **The phase in which each kind of MediQueue defect ought to have been detected**
- **Using the defect data table, the cost the hospital incurred because defects were discovered late**
- **The probable saving had requirements and design reviews been carried out**
- **The limitations of your estimate and the assumptions on which it rests**

Show your calculations and explain what they mean. Merely stating that "early testing is cheaper" will not earn full marks.

Evidence Expected

A process-oriented assessment of testing in MediQueue, backed by cost calculations from the defect data and a reasoned estimate of savings.

TASK 4 – Apply and Critique Basic Definitions 4 Marks

CO1 | Bloom's Level: K5 – Evaluate

In its reports, the team uses the terms "bug", "error" and "failure" as if they mean the same thing.
Does loose terminology genuinely damage quality, or is it merely a question of words?

Discuss:

- **What error, fault (defect), failure, test case, test oracle and test bed mean in the MediQueue context**
- **The error, the fault and the failure for incidents I3, I5 and I6 ●**

How the confusion over terminology allowed incident I8 to happen

- **Whether the team requires a shared glossary, and what it should include**

Definitions reproduced from a textbook will not earn full marks. Each one must be applied to MediQueue.

Evidence Expected

Accurate application of basic testing definitions to the MediQueue incidents and a justified judgement on how terminology affects quality.

TASK 5 – Evaluate the Application of Testing Principles 3 Marks *CO1 | Bloom's Level: K5 – Evaluate*

The developers assert that "we tested everything and found no errors, so the software was correct".

Which testing principles show that this assertion is false?

Discuss:

- **A minimum of four software testing principles relevant to the MediQueue situation**
- **For every principle, the incident or decision in the case that breaks or supports it**
- **Which principle, had it been followed, would have had the greatest impact on quality, and why**

Do not just list the principles. You must link each principle to

evidence. Evidence Expected

A justified assessment of the developers' assertion using testing principles tied to evidence from the case.

TASK 6 – Evaluate the Tester's Role 3 Marks

CO1 | Bloom's Level: K5 – Evaluate

MediQueue did not have an independent tester.

Would an independent tester have altered the outcome, or would the same defects still have reached patients?

Discuss:

- **The role and duties of a tester throughout the life cycle**
- **The benefits and drawbacks of developers testing their own code**
- **Which incidents a tester would most probably have detected, and at what stage**
- **The qualities required of a tester working on a healthcare application**

Your answer must adopt a position and defend it.

Evidence Expected

A defended judgement on how valuable an independent tester would be for MediQueue, tied to specific incidents.

TASK 7 – Trace the Origins of Defects 4 Marks

CO1 | Bloom's Level: K5 – Evaluate

Using the incidents logged after release, determine where each defect originated.

At which points in the life cycle are MediQueue's defects actually

originating? Discuss:

- **The origin of every incident: requirements, design, code, testing, documentation or environment**

- **The evidence from the case that supports each origin you assign**
- **Incidents that could have more than one origin, and how you reached your decision**
- **What the overall pattern of origins indicates to the hospital about where effort should be invested**

Where an incident could fit more than one origin, argue in favour of your choice. Evidence Expected

A justified origin for every incident and an interpretation of the overall pattern of defect origins.

TASK 8 – Evaluate Defect Classes and Severity 4 Marks

CO1 | Bloom's Level: K5 – Evaluate

A scheme for classifying defects helps the team understand what kind of defects it has.

Which defect classes cause the most harm to patients, and do the team's priorities match this?

Discuss:

- **An appropriate defect classification (requirements/specification, design, coding and testing defects together with their sub-classes)**
- **The class, severity and priority of every incident, with reasons**
- **Why the same defect may have different severity and priority (for example, I7 and I1)**
- **Which defect class the team ought to address first, and why**

Just placing defects in a table is not enough. You must justify every classification.

Evidence Expected

A justified classification of all ten incidents by class, severity and priority, together with a recommendation on which class to address first.

TASK 9 – Design and Appraise a Defect Repository 3 Marks
CO1 | Bloom's Level: K5 – Evaluate

At present the hospital logs complaints in e-mails and a spreadsheet.

What information should a defect repository store so that the team can learn from its defects and avoid repeating them?

Discuss:

- **The fields a MediQueue defect repository should include, and why each one is required**
- **How the repository supports defect analysis, process improvement and prevention**
- **The shortcomings of the present e-mail and spreadsheet**

method You must justify your repository design rather than merely

listing fields. Evidence Expected

A defect repository design for MediQueue with a justification for every field and an assessment of its benefits.

TASK 10 – Evaluate Defect Examples and Prevention Strategies 4 Marks

CO1 | Bloom's Level: K5 – Evaluate

Choose three incidents that belong to different defect classes. For each

chosen defect, which single process change would have prevented it?

Discuss:

- **The root cause of each chosen defect**
- **A prevention strategy for each one (for example, requirement reviews, design inspection, coding standards, test oracles)**
- **The cost and effort of each strategy weighed against its**

benefit ● Which strategy offers the hospital the best value

Your recommendation must be backed by evidence from the case and the defect data.
Evidence Expected

A root-cause analysis of three defects with justified prevention strategies that take cost into account.

TASK 11 – Recommend and Defend a Quality Improvement Plan 4 Marks

CO1 | Bloom's Level: K5 – Evaluate

Drawing on your analysis in Tasks 1–10, propose a quality improvement plan for MediQueue.

Discuss:

- **The process changes you propose, in order of priority**
- **How testers, reviews and the defect repository fit into your plan**
- **The expected impact on the number and cost of defects**
- **The trade-offs, risks and limitations of your plan**
- **Whether the plan is still workable for a small team of five developers**

Your recommendation must arise from your own analysis. A generic list of best practices will not be enough.

Evidence Expected

A technically justified quality improvement plan backed by the evidence built up in earlier tasks.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Quality Process 8 Marks

CO1 | Bloom's Level: K5 – Evaluate

Changed Requirement
The hospital now wishes to release a new MediQueue version every two weeks rather than every six months.

In addition, MediQueue has to connect to the national digital health ID system, which an external agency maintains.

Patient health records will from now on be shared across hospitals. Reassess your original solution in light of these changed conditions. You may:

RETAIN → MODIFY → or REDESIGN

your original solution.

A. Impact Analysis – 2 Marks

CO1 | Bloom's Level: K5 – Evaluate

Discuss:

- **What is different from the original situation**
- **Which assumptions in your quality improvement plan are impacted ● Why frequent releases alter the role of testing and of the tester**
- **Why sharing health records increases the severity of certain defect classes**

Evidence Expected

A clear analysis of how the new conditions impact the original quality improvement plan.

B. Evaluation of the Original Plan – 2 Marks

CO1 | Bloom's Level: K5 – Evaluate

Assess whether the plan you recommended is still suitable.

Discuss:

- **Whether the planned reviews and test activities can fit within a two week cycle**
- **Which new defect origins the external health ID system brings in**
- **Whether your defect repository is still adequate**

Evidence Expected

A technically supported assessment of how suitable the original plan is under the new conditions.

C. Adaptation or Redesign – 2 Marks

CO1 | Bloom's Level: K5 – Evaluate

Decide whether to:

Retain, modify or redesign the quality process.

If you modify or redesign it:

- **Describe the revised process and the revised role of the tester**
- **Explain which testing principles gain greater importance now**
- **Explain how defects coming from the external system will be classified and tracked**

If you keep the original plan, give adequate justification for why no change is needed.

Evidence Expected

A suitable response to the changed requirement with clear quality-engineering reasoning.

D. Evidence-Based Defence – 2 Marks

CO1 | Bloom's Level: K5 – Evaluate

Justify your final decision with evidence.

Use:

- **The defect data and the cost figures**
- **Defect origins and classes**
- **Release frequency**
- **Risks to patient safety and data privacy**
- **Team size and capacity**

Your final decision must follow logically from the analysis you have carried out. Evidence Expected

A well-supported final decision that brings together defect evidence, process reasoning and the changed constraints.

MARK DISTRIBUTION

Task	Marks
Task 1 – Evaluate Testing as an Engineering Activity	5
Task 2 – Evaluate the Role of Process in Software Quality	2

Task 3 – Evaluate Testing as a Process	**6**
Task 4 – Apply and Critique Basic Definitions	**4**
Task 5 – Evaluate the Application of Testing Principles	**3**
Task 6 – Evaluate the Tester's Role	**3**
Task 7 – Trace the Origins of Defects	**4**
Task 8 – Evaluate Defect Classes and Severity	**4**
Task 9 – Design and Appraise a Defect Repository	**3**
Task 10 – Evaluate Defect Examples and Prevention Strategies	**4**
Task 11 – Recommend and Defend a Quality Improvement Plan	**4**
Subtotal	**42**
Task 12 – Day-4 Design Challenge	**8**
TOTAL	**50**

SOFTWARE TESTING (CSA37)
ASSIGNMENT 1 – CO1

CO1 Coverage: Testing viewed as an engineering activity, the role of process in software quality, testing as a process, basic definitions, principles of software

testing, the role of the tester, origins of defects, defect classes, defect repository and defect examples.

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date: _____

Assignment Title: Evaluate the Quality Process and Defect Profile of a Hospital Appointment Application (Total Marks - 50)

Duration:1week

Scenario — Hospital Outpatient Appointment Application (MediQueue)

MediQueue is a mobile and web application introduced by a multi-speciality hospital. Through it, patients can book outpatient appointments, pay consultation fees, obtain token numbers and view their prescriptions. Roughly 6,000 patients use the application each day.

A team of five developers built the application over eight months, with no dedicated tester. The developers "tested" their own code shortly before release, and the release was signed off because the demonstration to hospital management showed no errors.

In the three months following release, the hospital received a large number of complaints. As the newly appointed Quality Engineer, management has asked you to find out why so many defects reached patients and to recommend how quality can be built into the process.

Incidents logged after release:

ID	Incident
I1	When two patients book at exactly the same second, both are issued the same token number for the same doctor.
I2	The system accepts Sunday appointments even though the outpatient department does not operate on Sundays; this rule was never captured in the requirements.
I3	The booking page displays a consultation fee of ₹500, yet ₹550 is collected; the GST rule was misinterpreted.
I4	The application crashes whenever a patient name is longer than 40 characters.
I5	Patients who reach the age of 60 on the appointment day do not receive senior-citizen priority.
I6	When two doctors save at the same moment, the prescription PDF displays the details of the previous patient.
I7	The user manual instructs patients to cancel via "My Visits", but no such menu exists.
I8	A refund test case was recorded as "Pass" because the tester checked the output against an incorrect expected value.
I9	SMS reminders stop being sent when the hospital's SMS provider changes its API key.
I10	On the hospital's Wi-Fi network, the doctor's schedule screen needs 25 seconds to load.

Defect data from the most recent release (100 defects):

Phase	Defects introduced	Defects detected	Cost to fix each defect (₹)	Remarks
Requirements	30	2	500	No formal review
Design	22	3	1,500	No formal review
Coding	38	15	3,000	Developer self-check only
Testing	5	20	8,000	Demo-based testing
Documentation	5	0	–	–
Production (after release)	–	60	40,000	Reported by patients

Design Challenge:

The hospital intends to release a new version every two weeks and to link MediQueue with the national digital health ID system.

Task 1 – Evaluate Testing as an Engineering Activity 5 Marks

CO1 | Bloom's Level: K5 – Evaluate

Examine whether the way MediQueue was tested qualifies as an engineering activity.

Did the MediQueue team deliberately engineer quality into the product, or did they simply hope it would be there?

Discuss:

- The features of testing as an engineering activity (planning, measurement, standards, repeatability)
- Which of these features were absent in the MediQueue project, supported by evidence from the case
- How the absence of each feature led to the incidents that were reported
- What an engineered approach to testing would have looked like for this project You are required to assess the project against engineering practice rather than just define the term.

Evidence Expected

A well-argued assessment of MediQueue's testing practice against the features of an engineering activity, backed by evidence from the case.

Ans:

Did MediQueue Testing Qualify as an Engineering Activity?

My aim is to judge whether the MediQueue team deliberately engineered quality into the product or merely hoped for it. To do so, I compare the team's actual practice against the features that make testing an engineering activity: it is planned, it is measured, it follows defined standards and another person can repeat it. My only evidence is the set of facts given in the case.

I conclude that MediQueue's testing was not an engineering activity. The developers "tested" their own code shortly before release, and approval was granted because a demo produced no errors. That is an informal approach rather than an engineered

process: a demo only proves that the paths the developers chose to show are working, and reveals nothing about the paths no one exercised.

1. PLAN	2. DEFINE STANDARDS	3. DESIGN TESTS	4. EXECUTE	5. MEASURE	6. IMPROVE
Risk and scope	Acceptance criteria	Test data and oracle	Unit to system	Coverage and defects	Root-cause actions
↓	↓	↓	↓	↓	↓

Figure 1. Engineering quality cycle for MediQueue

Table 1: MediQueue testing compared against the features of an engineering activity

Characteristic	Expectation of engineering practice	Evidence from the case	Incidents it led to
Planning	Test goals, test levels, schedule, resources and entry/exit criteria documented before testing begins	There is no test plan. Developers performed testing "just before release", after eight months of construction. No time or staff were allocated to it	I10, I9, I1, I6 (no performance, interface or concurrency test was ever planned)
Measurement	Coverage, defects found in each phase and cost are measured and used to make release decisions	The release criterion was "the demo showed no errors". Only 40 of the 100 defects were detected before release and patients found the other 60, yet there was no target or exit measure to show that this was unacceptable	All ten; this explains why 60% of the defects slipped through
Standards and defined process	Reviews of requirements and design, a standard test case format and traceability to requirements	Requirements and design received "no formal review"; coding had only a "developer self-check". Requirement rules (Sunday closure, GST) were never documented	I2, I3, I7
Repeatability	Documented test cases, test data and expected results that someone else can re-run after every change	Testing based on a demo cannot be repeated. The single documented case among the incidents (refund) was passed using an incorrect expected value	I8, I4, I5
Independence and defined roles	Someone whose job is to find defects and who has testing skills	There was no tester. Developers checked their own code, so the same mistaken assumptions were used both to write it and to "test" it	I5, I1, I6
Tools and infrastructure	Test environment, test data, defect tracking and automation for repeated tests	Complaints were stored in e-mails and a spreadsheet; there was no test bed for the SMS provider or for load	I9, I10

How the absence of each feature led to the incidents

- No planning: Nothing in the process prompted the question "what might fail on a slow network, with two users in the same second, or after an SMS key change?". I10, I9, I1 and I6 are precisely the defects that surface only when such tests are planned.
- No measurement: Without an exit criterion the team had no view of the risk. Before release, only 5 of the 52 requirement and design defects had been

detected (2 + 3), and the 20 defects found during testing were only about one quarter of the 75 defects present at that stage.

- Absence of standards or reviews: A missing business rule (I2) and a misread tax rule (I3) can only be caught by someone who checks the requirement with the people who own the rule. There was no review, so the rule was never verified. The incorrect menu name in the manual (I7) reflects the same absence of a document review.
- No repeatability: Without written test cases and expected values, boundary cases like a 41-character name (I4) or a patient aged exactly 60 (I5) are only checked by chance. When a test case was actually written (refund), its expected value was incorrect and no one re-verified it (I8).
- No independent tester: Whoever writes a piece of code carries the same assumptions into testing it. A developer convinced that “two doctors never save at the same time” (I6) would never write that test.

What an engineered approach to testing would have looked like

- A brief test policy plus a test plan for every release, covering goals, levels (unit, integration, system, acceptance) and a schedule.
- Release exit criteria, for example: every planned high-risk test executed; no open critical or high defect; at least 95% of planned tests passed.
- Reviews of the requirements (together with billing and OPD staff) and of the design (concurrency and data separation) before coding starts.
- Test cases built using equivalence classes and boundary values, each having an expected result drawn from the requirement and traced back to it.
- System tests for concurrency, performance on the hospital Wi-Fi and the SMS interface, executed in a test environment.
- A defect repository with simple measures (defects per phase, cost, escapes) reviewed at each release.

Task 2 – Evaluate the Role of Process in Software Quality 2 Marks

CO1 | Bloom's Level: K5 – Evaluate

Hospital management is convinced that employing "better developers" will fix the quality problem.

Does quality come mainly from good people or from a good process?

Discuss:

- How a defined process contributes to achieving software quality
- Whether better developers on their own would have prevented the incidents ● Where a process maturity view (for example, TMM levels) would position the current team

You must defend your position using evidence from the case.

Evidence Expected

A well-justified position on how process affects quality, backed by data from the case.

Ans:

Good People or Good Process?

LEVEL 1	LEVEL 2	LEVEL 3	LEVEL 4–5
Ad hoc	Managed basics	Integrated testing	Measured and optimised
Developer demo	Plans and defect log	Reviews + lifecycle tests	Metrics + prevention
CURRENT STATE	IMMEDIATE TARGET	NEXT TARGET	LONG-TERM GOAL

Figure 2. Testing maturity progression

My position is that MediQueue's quality problem is chiefly a process problem. Better developers would be useful, but they would not have stopped most of the incidents, since most defects arose before coding or outside the code altogether.

Table 2: Could a better developer on their own have prevented the incident?

Incident	Prevented by a better developer alone?	Reason
I1, I6	Possibly	An experienced developer might consider concurrency, but a design review and a concurrency test would still be required to be certain
I4, I5	Probably	Handling input length and age boundaries is a coding skill, yet a boundary test would catch these even with an average developer
I10	Partly	Also requires a performance target (requirement) and a load test
I2, I3	No	The Sunday rule was never documented and the GST rule was misread; coding skill cannot provide a rule that was never given to the developer
I7	No	A documentation defect; it requires a document review
I8	No	A defect in testing; it requires a test-case review and an independent tester
I9	No	The SMS provider altered its key; this requires interface monitoring and configuration management

Skill alone could have prevented at most 4 or 5 of the 10 incidents. The defect data confirms this: only 38 of the 100 defects were introduced during coding, whereas 52 came from requirements and design. Even if better developers cut coding defects in half, 19 defects would vanish and 81 would stay, so “hiring better developers” solves less than a fifth of the problem. A defined process (reviews, planned tests, an independent tester, a defect repository) acts on every phase and also catches the mistakes that good developers make.

The team’s position on a process maturity scale

Using the Testing Maturity Model (TMM), I place the team at Level 1 (Initial). At this level, testing is not distinguished from debugging, testing only aims to show that the software works (the demo), and there is neither a test plan nor a trained tester. The team should target Level 2 (Phase Definition), whose maturity goals match exactly what is lacking: define testing and debugging goals, begin test planning, and adopt basic testing techniques and methods. Level 3 (Integration) would come next: a test group, a test life cycle integrated with the development life cycle, and training.

Task 3 – Evaluate Testing as a Process 6 Marks

CO1 | Bloom's Level: K5 – Evaluate

In MediQueue, testing was carried out as one activity immediately before release.

How would the defect data have differed if testing had been treated as a process spanning the entire life cycle?

Discuss:

- Testing as a process made up of planning, design, execution, evaluation and reporting activities
- How verification and validation activities relate to each development phase ● The phase in which each kind of MediQueue defect ought to have been detected ● Using the defect data table, the cost the hospital incurred because defects were discovered late
- The probable saving had requirements and design reviews been carried out ● The limitations of your estimate and the assumptions on which it rests

Show your calculations and explain what they mean. Merely stating that "early testing is cheaper" will not earn full marks.

Evidence Expected

A process-oriented assessment of testing in MediQueue, backed by cost calculations from the defect data and a reasoned estimate of savings.

Ans:

Testing as a Process Across the Life Cycle

PHASE	VERIFICATION – BUILD IT RIGHT?	VALIDATION – BUILD THE RIGHT THING?
Requirements	Review rules and completeness	Hospital approves workflows
Design	Inspect architecture and data flow	Review prototype with staff
Coding	Peer review and unit tests	Check acceptance criteria
Integration	API and database tests	Validate booking-payment-SMS journey
System / release	Regression, security, performance	User acceptance and sign-off

Figure 3. Verification and validation across the life cycle

In MediQueue, testing happened as a single event before release. I aim to show what would be different if testing were a process running alongside development. First I outline the process for MediQueue, then I identify the phase in which each incident should have been caught, and finally I use the defect data to calculate the cost of late detection and the saving reviews would have brought.

1. Testing viewed as a process

Table 3: Test process activities as applied to MediQueue

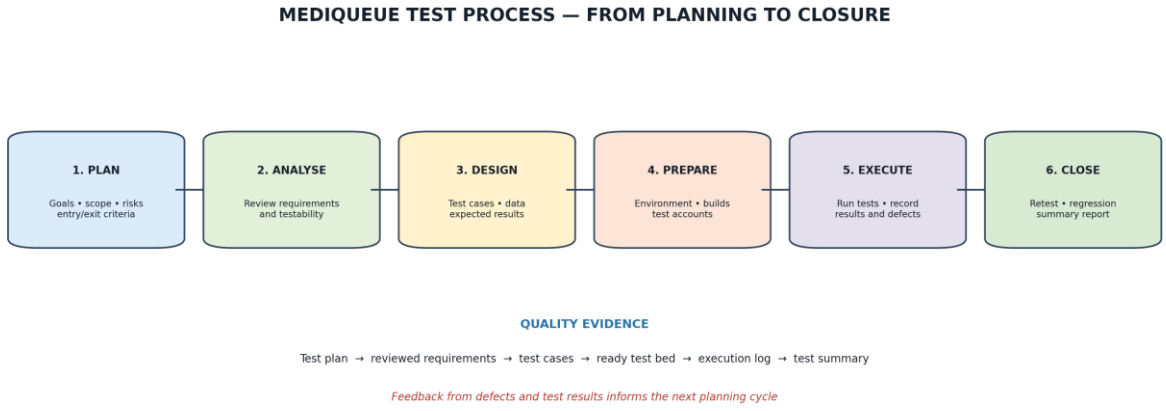


Figure 13. MediQueue testing activities from planning to closure

Figure 13. MediQueue testing activities from planning to closure

2. Verification and validation for each phase

Table 4: Verification and validation activities for each development phase

Phase	Verification (are we building it right?)	Validation (are we building the right thing?)	MediQueue defects targeted
Requirements	Requirement review using a checklist, plus a testability check	Acceptance test planned together with hospital staff	I2, I3, I10
Design	Design inspection (concurrency, data handling and interfaces)	System test cases are designed	I1, I6, I9
Coding	Code review, coding standard and unit tests using boundary values	–	I4, I5
Integration and system testing	Interface tests, regression tests and test-case review	System tests (performance, concurrency) and user acceptance test	I1, I6, I9, I10, I3
Documentation	Manual verified against the working system	Manual tried out by a real user	I7
Operation	Interface monitoring and regression after changes	Patient feedback	I9

3. The phase in which each incident ought to have been detected

Table 5: Intended detection phase for each incident

Incident	Defect type	Phase in which it should have been detected	Activity that would reveal it
I1	Two patients get the same token	Design	Design inspection (concurrency); followed by a concurrent-booking system test
I2	Sunday rule missing	Requirements	Requirement review together with OPD staff
I3	₹500 fee displayed, ₹550 charged	Requirements	Requirement review with billing; a system test comparing the displayed and charged fee
I4	Crash for names longer than 40 characters	Coding	Unit test using boundary values (40, 41)
I5	Senior priority at the age of 60	Coding	Boundary test (59, 60, 61); code review
I6	Prescription PDF shows the wrong patient	Design	Design inspection (shared data); a concurrent-save system test
I7	Manual refers to a menu that does not exist	Documentation	Review of the document against the application

I8	Incorrect expected value in the refund test	Testing	Review of the test case by a second person
I9	SMS stops working after the key change	Design / integration	Interface testing and configuration management; monitoring in production
I10	Schedule screen needs 25 seconds	Requirements / system test	Performance requirement; load testing on the hospital Wi-Fi

Under this plan, five of the ten incidents (I1, I2, I3, I6 and I10) fall under requirements and design, two (I4, I5) under coding, and the remaining three under testing, documentation and operation. In the actual project, patients found all ten.

4. The cost of late detection to the hospital

Table 6: Cost of the last release's 100 defects, based on the cost per defect given in the case

Phase of detection	Defects detected	Cost for each defect (₹)	Total cost (₹)	Proportion of total cost
Requirements	2	500	1,000	0.0%
Design	3	1,500	4,500	0.2%
Coding	15	3,000	45,000	1.7%
Testing	20	8,000	1,60,000	6.1%
Production (after release)	60	40,000	24,00,000	91.9%
Total	100	avg 26,105	26,10,500	100%

Sample calculation: production = 60 × ₹40,000 = ₹24,00,000; testing = 20 × ₹8,000 = ₹1,60,000; coding = 15 × ₹3,000 = ₹45,000; design = 3 × ₹1,500 = ₹4,500; requirements = 2 × ₹500 = ₹1,000. The total comes to ₹26,10,500, an average of ₹26,105 per defect.

Figure 1: Defect cost by the phase in which the defects were detected

Interpretation: Patients found 60% of the defects, and these account for 91.9% of the total cost. A single production defect costs 80 times as much as a requirements defect (40,000 ÷ 500) and five times as much as one found in testing (40,000 ÷ 8,000). Had the 60 production defects been caught in testing at ₹8,000 each, they would have cost ₹4,80,000 rather than ₹24,00,000, a difference of ₹19,20,000. This is an upper limit, since testing could not have caught every one of them. Had each defect been fixed in the phase that introduced it, the 95 defects with a unit cost would have cost 30 × 500 + 22 × 1,500 + 38 × 3,000 + 5 × 8,000 = ₹2,02,000. Late detection therefore cost roughly ₹24 lakh (₹26,10,500 − ₹2,02,000), around twelve times the ideal cost.

Table 7: Number of defects caught by each phase out of those present in that phase

Phase	Defects introduced	Defects present in this phase	Defects detected	Detection efficiency	Defects passed on
Requirements	30	30	2	6.7%	28

Design	22	50	3	6.0%	47
Coding	38	85	15	17.6%	70
Testing	5	75	20	26.7%	55

Phase	Defects introduced	Defects present in this phase	Defects detected	Detection efficiency	Defects passed on
Documentation	5	60	0	0.0%	60
Production	–	60	60	100.0%	0

This table tracks each defect through the phases (defects present = defects carried over from the previous phase + new defects; the final column feeds into the next row, and the 60 defects remaining after documentation equal the 60 found in production). Requirements and design, where 52 defects were introduced, detected only 5 (9.6%). Only 40 of the 100 defects were found before release (40% detection), so the "no errors in the demo" decision was taken while 60 defects still lay hidden in the product.

5. Probable saving had requirements and design reviews been conducted I modelled reviews using the flow in Table 7. I assume that a requirements review and a design review each remove a fraction r of the defects present in their phase (in place of the current 6.7% and 6.0%), and that coding and testing retain their existing detection rates (17.6% and 26.7%). Defects removed earlier are fixed at that phase's cost, and the remainder move on to later phases. I tested r = 40%, 60% and 80%, and assumed ₹1,00,000 as the cost of running both review programmes (roughly 125 person-hours at ₹800 per hour, including preparation and meetings).

Table 8: Estimated savings from requirements and design reviews

Review efficiency	Defects detected by the reviews	Defects reaching patients	Total cost after the reviews (₹)	Gross saving (₹)	Review cost (₹)	Net saving (₹)	Net saving as a % of ₹26,10,500
40%	28.0	46.1	20,26,808	5,83,692	1,00,000	4,83,692	18.5%
60%	38.4	39.8	17,61,399	8,49,101	1,00,000	7,49,101	28.7%
80%	46.4	35.0	15,55,854	10,54,646	1,00,000	9,54,646	36.6%

With reviews that are 60% efficient, defects reaching patients drop from 60 to around 40, cost drops from ₹26,10,500 to around ₹17,61,000, and the net saving is about ₹7.5 lakh (28.7%) after paying for the reviews. Reviews pay off even at 40% efficiency (net ₹4.8 lakh), because ₹1 lakh spent on reviews avoids many ₹40,000 production fixes. In the same model, a test process catching 60% of defects present in the testing phase

(rather than 26.7%) would cut production defects from 60 to 35 and save roughly ₹8 lakh, which shows that reviews and better testing complement each other.

6. Limitations of the estimate and assumptions

- Defect flow: The case does not state which phase each detected defect originated from, so I assumed the flow in Table 7 (defects missed in one phase move to the next). The actual flow could be different.
- Review efficiency: The 40–80% range is an assumption, as the team has no review data. Badly conducted reviews would fall at the lower end.
- Unit costs: I used the per-defect cost provided in the case, which covers rework only. It excludes patient harm, lost trust, legal exposure and hospital staff time, so the true cost of production defects is higher.
- Documentation defects: The case provides no cost for them; I treated the five left at the end as production defects.
- Single release, single team: The data come from one release of one project with 100 defects; defects that were never discovered are not included.
- Review cost: The ₹1,00,000 figure is my own estimate. A much higher cost would lower the benefit but not eliminate it.

Task 4 – Apply and Critique Basic Definitions 4 Marks

CO1 | Bloom's Level: K5 – Evaluate

In its reports, the team uses the terms "bug", "error" and "failure" as if they mean the same thing.

Does loose terminology genuinely damage quality, or is it merely a question of words?

Discuss:

- What error, fault (defect), failure, test case, test oracle and test bed mean in the MediQueue context
- The error, the fault and the failure for incidents I3, I5 and I6
- How the confusion over terminology allowed incident I8 to happen
- Whether the team requires a shared glossary, and what it should include

Definitions reproduced from a textbook will not earn full marks. Each one must be applied to MediQueue.

Evidence Expected

Accurate application of basic testing definitions to the MediQueue incidents and a justified judgement on how terminology affects quality.

Ans:

Applying and Critiquing the Basic Definitions

HUMAN ERROR	FAULT / DEFECT	EXECUTED CONDITION	FAILURE
Wrong assumption	Incorrect requirement, design or code	Input or event activates fault	Observable incorrect behaviour
GST misunderstood	Wrong fee formula	Patient books and pays	₹500 shown; ₹550 charged (I3)

Figure 4. Error–fault–failure relationship

I will demonstrate that terminology is more than just words. If "bug", "error" and "failure" all describe the same thing, the team cannot tell where the problem lies (with a person, in the product, or in what the user experiences), and so cannot trace or prevent its cause. I apply the definitions first to MediQueue and then to the three incidents.

Table 9: Basic definitions in the context of MediQueue

Term	Meaning	MediQueue example
Error	A mistake committed by a person	A developer assumes only one doctor saves a prescription at any one time
Fault (defect, bug)	The flaw in the software or document caused by an error	The PDF generator stores the patient's data in a variable that all doctors share
Failure	The incorrect behaviour a user observes when the faulty code executes	A prescription PDF displays the details of the previous patient
Test case	The inputs, execution conditions and expected result for a single check	Book a Sunday appointment → the system must reject it and show "OPD closed"
Test oracle	The source used to determine the expected result (requirement, rule, calculation or expert)	The approved fee table states consultation = ₹500 + tax; this determines whether ₹550 is correct
Test bed	The environment where tests are executed: hardware, network, data, tools and stubs	A copy of the application with test doctors and patients, an SMS sandbox and the hospital's Wi Fi conditions

Table 10: Error, fault and failure for incidents I3, I5 and I6

Incident	Error (human mistake)	Fault (defect within the product)	Failure (what the user observed)
I3	The developer or analyst misread the GST rule and implemented it differently in two places	The booking page displays the base fee while the payment module adds tax: two separate fee calculations	The page displays ₹500 but the patient pays ₹550
I5	The developer interpreted "senior citizen" as "over 60", or compared only the year rather than the full date	A condition like age > 60 (or age calculated on the booking date rather than the appointment date)	A patient turning 60 on the appointment day is not given priority
I6	The developer assumed that two doctors would never save simultaneously	Patient details are kept in shared data, which the second doctor overwrites before the PDF is generated	The PDF displays the details of the previous patient

How terminology confusion allowed I8 to happen

In I8, a refund test was recorded as "Pass" because the tester checked the output against an incorrect expected value. The tester made an error that introduced a fault into the test case (a wrong oracle), which led to a false verdict. A team that calls everything a "bug" does not distinguish a fault in the product from a fault in the test, and never asks "where did this expected value come from?". Since no test oracle had been defined, the expected value was written from memory rather than from the requirement, and the "Pass" was trusted. As a result the refund defect stayed hidden, so the wrong word concealed a wrong test.

Does loose terminology damage quality?

Yes, in four ways. (1) Root-cause analysis breaks down: a failure is caused by a fault and a fault is caused by an error, and each calls for a different remedy (a code fix, a review checklist, training). (2) Measurement is distorted: the case's defect count (100 defects) blends failures reported by patients with faults in code, so one fault may be counted several times. (3) Communication with hospital management suffers, as "bug" conveys nothing about severity. (4) Fix verification is weak, because no one records which test case and oracle demonstrate that the fault has been removed.

A common glossary

The team requires a one-page glossary, agreed upon by developers, testers and hospital staff and maintained alongside the defect repository.

Table 11: Minimum contents of the MediQueue testing glossary

Group	Terms to define	Reason it is needed
Defect terms	Error, fault/defect, failure, incident	Distinguish cause, flaw and symptom in each report

Group	Terms to define	Reason it is needed

Test terms	Test case, expected result, test oracle, test bed, test log and regression test	Make tests repeatable and avoid incorrect expected values (I8)
Rating terms	Severity and priority levels with examples drawn from MediQueue	End disputes about what to fix first
Defect states	New, assigned, fixed, verified, closed and reopened	Track each defect through to closure
Product terms	Build, release, hotfix, appointment day, senior citizen, fee and token	Clear up ambiguities such as “senior at 60” (I5)

Task 5 – Evaluate the Application of Testing Principles 3 Marks

CO1 | Bloom's Level: K5 – Evaluate

The developers assert that "we tested everything and found no errors, so the software was correct".

Which testing principles show that this assertion is false?

Discuss:

- A minimum of four software testing principles relevant to the MediQueue situation ● For every principle, the incident or decision in the case that breaks or supports it ● Which principle, had it been followed, would have had the greatest impact on quality, and why

Do not just list the principles. You must link each principle to evidence.

Evidence Expected

A justified assessment of the developers' assertion using testing principles tied to evidence from the case.

Ans:

Which Testing Principles Show That the Developers' Claim Is Wrong?

PRINCIPLE	CASE EVIDENCE	TESTING RESPONSE
Testing shows defects, not absence	Demo passed; I1–I10 escaped	Use broader risk coverage
Exhaustive testing impossible	6,000 users and many combinations	Prioritise boundaries and combinations
Early testing	30 requirement + 22 design defects	Review before coding
Defect clustering	Booking, payment and prescription risks	Focus high-risk modules
Pesticide paradox	Repeated demo missed new risks	Refresh regression tests
Context-dependent testing	Healthcare and patient-data risks	Prioritise privacy and correctness

Figure 5. Testing principles linked to MediQueue evidence

The developers state that "we tested everything and found no errors, so the software was correct". The statement makes two mistakes: it treats the absence of failures in the tests actually run as proof of correctness, and it assumes that every possible test was run. I apply six testing principles and connect each to evidence from the case.

Table 12: Testing principles as applied to MediQueue

Principle	Violated or confirmed	Supporting evidence from the case
Testing has to be planned	Violated	There was no test plan; testing took place just before release, so no time or tests were set aside for load, concurrency or the SMS interface (I10, I1, I6, I9)
Testing activities should be built into the entire life cycle (start early)	Violated	Requirements and design were never reviewed; 52 of the 100 defects were introduced there and only 5 were caught early. Late detection cost ₹26.1 lakh, 91.9% of it in production
Test cases have to be written for valid as well as invalid inputs	Violated	A 41-character name (I4) and an appointment on a Sunday when the OPD is closed (I2) are invalid or extreme inputs that the demo never exercised

Principle	Violated or confirmed	Supporting evidence from the case

Every test case must state its expected output, and test results must be examined carefully	Violated	The refund test carried an incorrect expected value and was passed (I8)
Testing exists to expose defects; a good test case is one with a high probability of finding a defect not yet known	Violated	The "test" was a management demo intended to show success. It found nothing because it was never designed to find anything
Testing should be assigned to the most qualified and independent person	Violated	Developers tested their own code, so the same assumption (single user, age > 60) showed up in both the code and the tests (I6, I5)
Where one defect is found, others are likely to be close by (defects cluster)	Confirmed	I1 and I6 both arise from concurrent use of shared data; after one such defect was found, the other should have been searched for

Beyond these, a frequently quoted testing principle states that testing can reveal the presence of defects but never prove their absence. The developers treated the lack of failures in a handful of demo cases as proof of correctness, which is precisely what this principle rules out. In addition, exhaustive testing is not possible, so tests must be selected using risk and technique, and no such selection took place.

The principle with the greatest impact

In my judgement, "testing should be built into the entire life cycle and start early" would have had the greatest impact. Half the incidents (I2, I3, I10, I1, I6) originate in requirements and design, and the defect data show that these phases catch only 9.6% of their own defects, while the cost per defect climbs steeply with every phase (₹500 in requirements up to ₹40,000 in production). Planning (the first principle) and independence would help, but both are far more effective when they begin in the requirements phase. Writing test cases from the requirements would have revealed the missing Sunday rule and the fee mismatch before a single line of code was written.

Task 6 – Evaluate the Tester's Role 3 Marks

CO1 | Bloom's Level: K5 – Evaluate

MediQueue did not have an independent tester.

Would an independent tester have altered the outcome, or would the same defects still have reached patients?

Discuss:

- The role and duties of a tester throughout the life cycle
- The benefits and drawbacks of developers testing their own code
- Which incidents a tester would most probably have detected, and at what stage ● The qualities required of a tester working on a healthcare application

Your answer must adopt a position and defend it.

Evidence Expected

A defended judgement on how valuable an independent tester would be for MediQueue, tied to specific incidents.

Ans:

Would an Independent Tester Have Made a Difference?

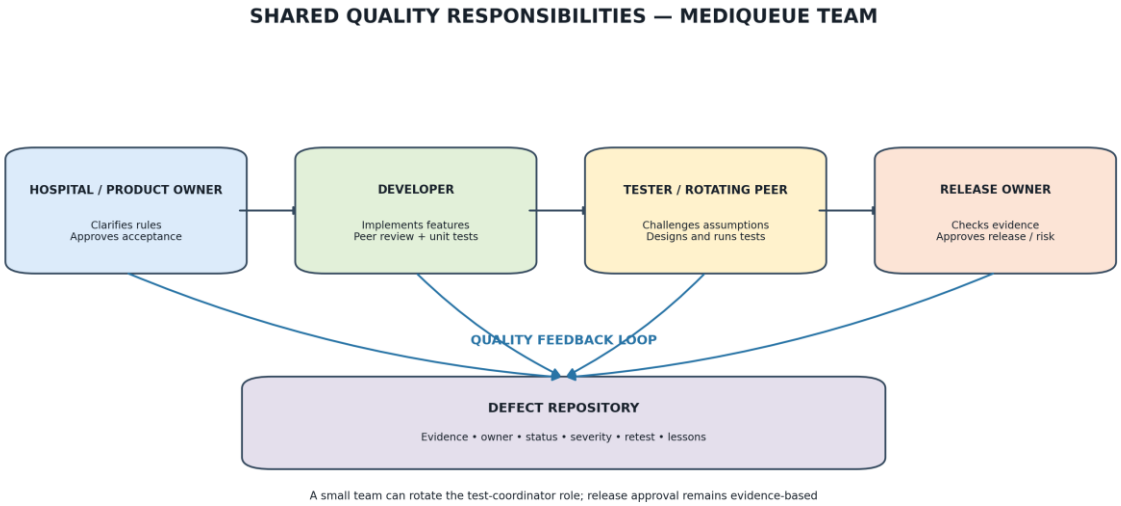


Figure 14. Shared quality responsibilities across the MediQueue team

Figure 6. Shared quality responsibilities

I hold that an independent tester would have changed the outcome considerably, though not eliminated every defect. A tester working across the life cycle would very probably have caught seven of the ten incidents; the remaining three (I1, I6, I9) also require design review, specialised tests and production monitoring.

The tester's role throughout the life cycle

- Requirements: review for statements that are missing, unclear or untestable; ask "what happens on a Sunday? what is the fee inclusive of tax? how quickly must the schedule load?".
- Design and planning: prepare the test plan, risks and exit criteria; design the test cases and the test bed.
- Coding and integration: support unit testing, execute integration tests and maintain the regression suite.
- System and acceptance: execute system tests (functional, load, concurrency, security) and support user acceptance.
- Throughout: report defects with reproduction steps, verify fixes, maintain the repository and provide management with evidence for the release decision.

Developers testing their own code

Benefits: the developer understands the code, spots simple faults fast and fixes them cheaply, and developer unit testing is valuable and should carry on. Drawbacks: the developer tests what he or she thinks the code does, so a mistaken assumption is carried into the test; there is a natural desire to show the work is good; deadlines push testing to the very end; and no one checks how the developer has interpreted the requirement. MediQueue illustrates this drawback: I5 (the age-60 boundary) and I6 (the single-user assumption) both stem from the developer's own assumptions.

Table 13: Incidents a tester would most probably have caught

Incident	Would a tester have detected it?	Stage and how
I2	Yes	Requirement review / test design: asks what the OPD working days are
I3	Yes	Requirement review; a system test comparing the fee displayed with the amount charged
I4	Yes	Unit or system test using boundary-length names (40 and 41 characters)
I5	Yes	Test design using boundary values of 59, 60 and 61 years on the appointment day
I7	Yes	Documentation review: follow the manual step by step on the actual application
I8	Yes	Second-person review of the test case; the oracle is traced back to the requirement
I10	Yes	System test including a load test on the hospital Wi-Fi once a performance target is agreed

Incident	Would a tester have detected it?	Stage and how
I1	Maybe	Only if a concurrency test is designed (this needs tools and skill); design inspection is the stronger safeguard
I6	Maybe	As with I1; requires a concurrent-save test and a design review of shared data
I9	Partly	Interface test with the SMS sandbox; the post-release key change requires monitoring and configuration control

The result is 7 "yes", 2 "maybe" and 1 "partly". In terms of the defect data, had a tester lifted the testing phase from catching 26.7% of defects present to 60% (Task 3), production defects would have dropped from 60 to roughly 35. That is a significant effect, but with 35 still reaching patients a tester on their own is insufficient, which is why my plan also includes reviews.

Qualities needed in a tester for a healthcare application

- **Domain knowledge:** knowledge of the outpatient workflow, fees and taxes, senior citizen rules and prescriptions.

- Eye for detail and an enquiring mind: spots a ₹50 discrepancy, an incorrect menu name or a wrong expected value.
- Sensitivity to patient safety and privacy: understands that revealing one patient's data to another (I6) is a serious breach.
- Technical skill: test design techniques, concurrency and load testing, defect tracking and basic automation.
- Communication and ethics: reports defects clearly and without assigning blame, keeps health data confidential and can say "not ready for release".

Task 7 – Trace the Origins of Defects 4 Marks

CO1 | Bloom's Level: K5 – Evaluate

Using the incidents logged after release, determine where each defect originated. At which points in the life cycle are MediQueue's defects actually originating?

Discuss:

- The origin of every incident: requirements, design, code, testing, documentation or environment
- The evidence from the case that supports each origin you assign
- Incidents that could have more than one origin, and how you reached your decision ● What the overall pattern of origins indicates to the hospital about where effort should be invested

Where an incident could fit more than one origin, argue in favour of your choice.

Evidence Expected

A justified origin for every incident and an interpretation of the overall pattern of defect origins.

Ans:

Where Are the MediQueue Defects Coming From?

REQUIREMENTS	DESIGN	CODING	TESTING / DOCS	ENVIRONMENT
I2: Sunday rule	I1: duplicate token	I4: long-name crash	I8: wrong oracle; I7: manual	I9: API key
I3: GST rule	I6: data isolation	I10: slow screen	Review gap	External dependency
Stakeholder review	Design inspection	Standards + BVA	Independent review	Contract tests

Figure 7. Defect-origin cause map

For each incident I assign a single main origin, cite the evidence from the case, and indicate where a second origin is possible and why I preferred the main one.

Table 14: Origin of every incident

Incident	Main origin	Supporting evidence from the case	Possible second origin and the reason for my choice
I1	Design	Two patients receive the same token only when they book in the same second; token allocation was not designed as an atomic operation	Coding. Design was chosen because the fault lies in how token allocation is structured rather than in a typing or logic slip
I2	Requirements	"This rule was never written in the requirements"	None
I3	Requirements	"The GST rule was misunderstood"; the rule was incorrect or unclear at the requirements stage	Coding. Requirements was chosen because the misunderstanding concerns the business rule, which is requirement knowledge
I4	Coding	Crash for names over 40 characters: length handling is missing	Requirements (no maximum name length was specified). Coding was chosen because the program must not crash on any input, whatever the specification says
I5	Coding	The "age 60" rule is straightforward; only the boundary was coded incorrectly	Requirements (the date on which the patient "turns 60" is not stated). Coding was chosen because the rule itself is common knowledge
I6	Design	When two doctors save at the same time they overwrite shared data; this is a design decision about shared state	Coding. Design was chosen because data sharing is an architectural decision

I7	Documentation	The manual refers to a "My Visits" menu that does not exist	None
I8	Testing	The tester checked the result against an incorrect expected value	None
I9	Environment / design	The SMS provider changed its API key; the application neither handled nor monitored an external change	Design (no error handling or alerting). The main origin is the environment because the trigger lies outside the team, but the design weakness let it cause harm
I10	Requirements	Nothing in the case indicates a performance target	Design or coding (inefficient loading). Requirements was chosen because without a target no one could judge 25 seconds to be a defect

Table 15: Overall pattern of origins

Origin	**Incidents**	**Count**	**Proportion of the 10 incidents**
Requirements	I2, I3, I10	3	30%
Design	I1, I6	2	20%
Coding	I4, I5	2	20%
Testing	I8	1	10%
Documentation	I7	1	10%
Environment	I9	1	10%

Interpretation: 5 of the 10 incidents (50%) arise from requirements and design, which matches the defect data for the last release (52 of 100 defects were introduced in these two phases). Only 2 incidents (20%) arise from coding, yet the team focused its (self-check) effort on coding alone. Three of the ten incidents are not code defects at all (testing, documentation, environment). Two incidents (I1, I6) share a single cause, shared data under simultaneous use, pointing to a design weakness that may also exist elsewhere.

What this means for the hospital: investment should go first into the early phases (requirement reviews and design inspections) and into an independent test activity, not into more coding skill. These are also the cheapest points at which to remove defects (₹500 and ₹1,500 per defect compared with ₹40,000 in production).

Task 8 – Evaluate Defect Classes and Severity 4 Marks

CO1 | Bloom's Level: K5 – Evaluate

A scheme for classifying defects helps the team understand what kind of defects it has. Which defect classes cause the most harm to patients, and do the team's priorities match this?

Discuss:

- An appropriate defect classification (requirements/specification, design, coding and testing defects together with their sub-classes)
- The class, severity and priority of every incident, with reasons
- Why the same defect may have different severity and priority (for example, I7 and I1) ● Which defect class the team ought to address first, and why

Just placing defects in a table is not enough. You must justify every classification.

Evidence Expected

A justified classification of all ten incidents by class, severity and priority, together with a recommendation on which class to address first.

Ans:

Defect Classes, Severity and Priority

	LOW PRIORITY	HIGH PRIORITY
CRITICAL / HIGH	Plan controlled fix	I6 data leak; I1 duplicate token; I3 fee mismatch
MEDIUM / LOW	I7 manual correction	I2 Sunday; I10 slow screen; I9 SMS
DECISION RULE	Severity = impact	Priority = urgency and exposure

Figure 8. Severity and priority decision matrix

The classification uses four main classes (requirements/specification, design, coding and testing defects) along with their standard sub-classes. Since two incidents fall outside these four classes (the user manual and the external SMS provider), I add a documentation class and treat the environment as an external-interface sub-class of design.

Table 16: Defect classification scheme applied to MediQueue

Class	Sub-classes	MediQueue examples
Requirements / specification	Functional description, feature, business rule, interface description and non-functional (performance) requirement	I2, I3, I10
Design	Algorithmic/processing, control logic and sequence, data and shared state, module interface and external interface	I1, I6, I9
Coding	Algorithmic/processing, control logic (boundary), data/input handling, initialisation, typographical and code documentation	I4, I5
Testing	Test-case design, test oracle (expected result), test procedure and test harness	I8
Documentation (added)	User manual, help text and release notes	I7

Scale applied: Severity is the impact of the defect on patients and the hospital: Critical = a risk to patient safety or privacy, or financial loss for many patients; High = a key function is wrong or unusable; Medium = the function is wrong but a workaround exists; Low = cosmetic or informational only. Priority is the order in which the team should fix it, based on severity combined with frequency, workaround, cost of the fix and the risk of delay.

Table 17: Class, severity and priority for every incident

Incident	Class / sub-class	Severity	Priority	Reason
I1	Design / control logic (concurrency)	High	Medium	Two patients sharing one token disrupt the queue, but the clash requires the same second and the same doctor, and reception can issue a new token, so it can come after the urgent items
I2	Requirements / business rule (missing)	Medium	High	Patients travel to an OPD that is closed, but safety is not affected; the fix is an inexpensive rule and many people are affected, so it is done early
I3	Requirements / business rule (wrong)	High	Critical (first)	Patients are overcharged, losing money and trust; there may be legal and tax implications; it recurs on every booking
I4	Coding / input handling	High	Medium	The application crashes, but only for names longer than 40 characters; a workaround (a shorter name) exists and the fix is easy
I5	Coding / control logic (boundary)	Medium	Medium	Patients turning 60 on the day lose their priority; the group is small, but it is a matter of fairness and care
I6	Design / data and shared state	Critical	Critical (first)	One patient's prescription appears with another patient's details: a breach of patient safety and privacy
I7	Documentation / user manual	Low	High	Data is not affected, but every patient who follows the manual is misled; the fix takes minutes, so it is done immediately
I8	Testing / test oracle	High	High	The false "Pass" conceals a possible refund defect and casts doubt on all test results; money is at stake
I9	Design / external interface	Medium	Medium	Missed reminders lead to missed appointments but no harm; requires alerting and key handling
I10	Requirements / non functional (performance)	Medium	Medium	The doctors' screen is slow but accurate; it needs a target and profiling, so it is planned for the next cycle

Why severity and priority may not match

Severity describes the damage caused if the defect occurs; priority describes how quickly it should be fixed, which also depends on how often it happens, whether a workaround exists, how cheap the fix is and what else is waiting. I7 is Low in severity

but High in priority: it damages no data, yet every patient reading the manual is affected and the fix takes minutes, making it a very cheap, immediate improvement. I1 is High in severity but only Medium in
priority: the impact is serious, but it occurs only when two bookings land in the same second and reception can issue a new token, so with I3 and I6 waiting, it comes after them. This pair also demonstrates that a priority list based on severity alone would be incorrect.

The most harmful class and where to begin

The defects most harmful to patients are the design defects in shared data (I6, Critical) and the requirement defect involving money (I3, Critical). The team's own priorities do not reflect this: it considered only visible errors during a demo and had no severity scale, so a crash and a privacy leak would have been handled identically.

I recommend the team addresses the requirements/specification class first, for three reasons: (1) this class accounts for 30 of the last release's 100 defects and 3 of the 10 incidents; (2) only 2 of those 30 defects were caught before release, a detection efficiency of 6.7%; (3) it is the cheapest class to fix early (₹500 per defect versus ₹40,000 in production). Design inspection of shared data and concurrency should follow straight after, since that class contains the most serious (Critical) defect, I6.

Task 9 – Design and Appraise a Defect Repository 3 Marks

CO1 | Bloom's Level: K5 – Evaluate

At present the hospital logs complaints in e-mails and a spreadsheet.

What information should a defect repository store so that the team can learn from its defects and avoid repeating them?

Discuss:

- The fields a MediQueue defect repository should include, and why each one is required
- How the repository supports defect analysis, process improvement and prevention ● The shortcomings of the present e-mail and spreadsheet method

You must justify your repository design rather than merely listing fields.

Evidence Expected

A defect repository design for MediQueue with a justification for every field and an assessment of its benefits.

Ans:

A Defect Repository for MediQueue

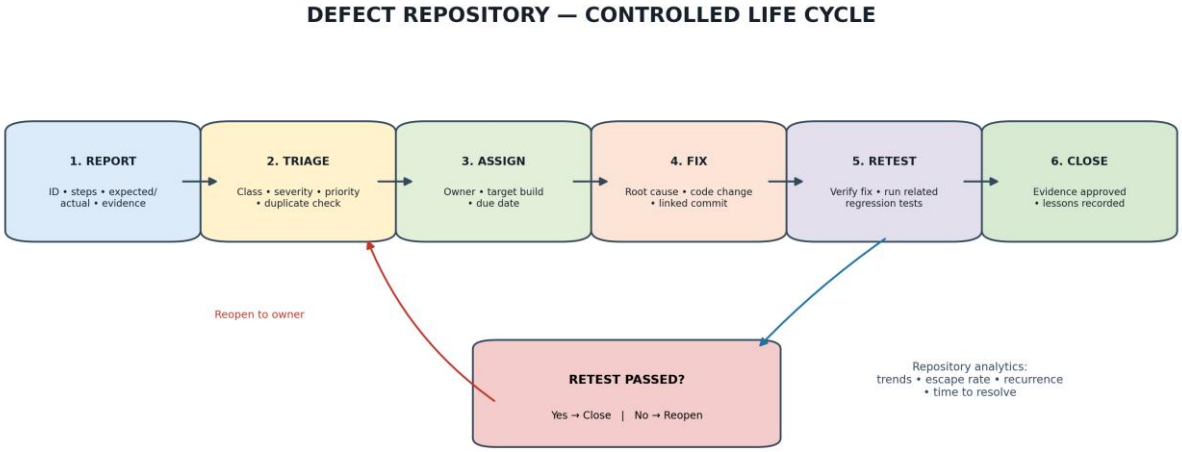


Figure 9. Defect repository workflow

A defect repository serves as the project's memory. It has to hold enough information about each defect for the team to answer: what went wrong, where it began, why it was not caught earlier, how serious it was, how it was fixed and how to prevent it from recurring. Below I design the MediQueue repository and justify each group of fields.

Table 18: MediQueue defect repository fields and the reason for each

Field group	Fields	Reason it is needed
Identification	Defect ID, title, date reported, reporter and source (patient complaint, tester, review, monitoring)	A unique reference; shows who finds defects and how, so that weaknesses in detection become visible
Description	Description, reproduction steps, expected result, actual result and attachments	Allows a developer to reproduce and fix the defect without further questions; distinguishes the failure from expected behaviour
Location	Module/feature, release and build number, environment (production, test) and browser/device	Shows where defects cluster and which release contains them
Classification	Class and sub-class (requirements, design, coding, testing, documentation, environment)	Basis for analysing types of defects and for building checklists

Origin and detection	Phase introduced, phase detected and detection method (review, unit test, system test, production)	Provides the phase-containment figures used in Task 3; shows the escape rate
Rating	Severity, priority, patient-safety/privacy flag and number of patients affected	Sets the order of work; highlights serious risks
Handling	Status (new, assigned, fixed, verified, closed, reopened), owner and dates	Tracking through to closure and ageing reports
Cause and fix	Root cause (error), description of the fix, fix effort (hours) and cost	Enables cost-per-phase figures and learning from causes
Traceability	Related requirement ID, test case ID and related defects/duplicates	Reveals which requirement or test was missing and links recurring defects
Prevention	Proposed preventive action, owner and status	Converts every defect into a process improvement

How the repository supports analysis, improvement and prevention

- Defect analysis: Pareto charts of defects by class, module and phase show where the team should focus; counts of origin and detection phase supply the figures used in Task 3 for every release.
- Process improvement: An increasing share of requirement defects found late proves that the review is ineffective; a high number of reopened defects points to weak fixing or verification; the data are used to set targets (for example, production escapes below 20%).
- Prevention: Every cause is converted into an action (a checklist item, a coding rule, a new regression test). For instance, I1 and I6 would result in a design checklist entry “shared data under simultaneous use” and a concurrency test for each release.
- Regression testing and test design: The defect list provides regression test cases and fresh test ideas for similar modules.
- Management: Supplies trend figures for release decisions in place of “the demo went well”.

Shortcomings of the current e-mail and spreadsheet method

Table 19: E-mail and spreadsheet versus a repository

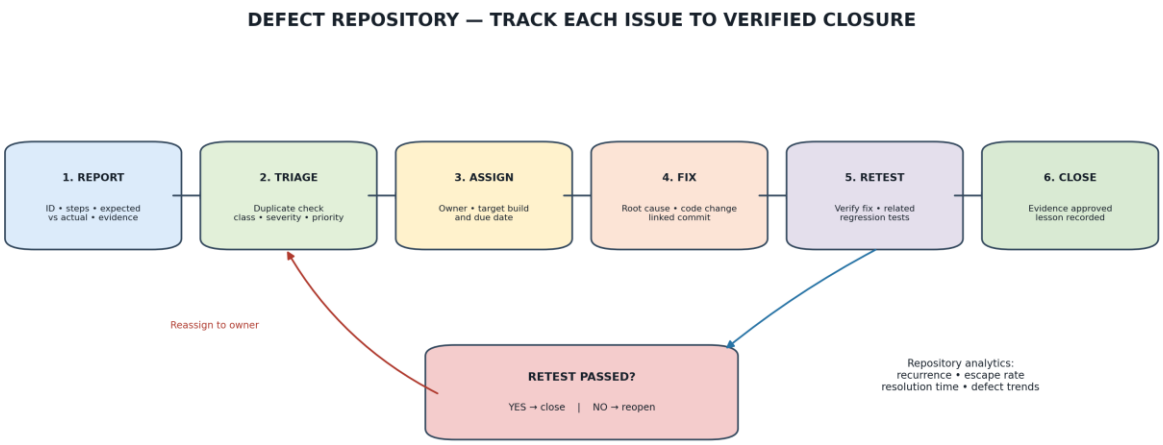


Figure 15. Defect repository workflow from report to verified closure

Figure 14. Why a central defect repository is stronger than e-mail and spreadsheets

Aspect	**E-mails and spreadsheet**	**Defect repository**
Workflow	No status or owner; defects are lost in mailboxes	Status, owner, due dates and reminders
History and audit	No change history; versions of the sheet conflict with one another	Each change logged with date and user
Analysis	Difficult to count by class, phase or cost without manual effort	Built-in reports and filters
Traceability	No link to the requirement, test case or build	Linked to all three
Privacy	Patient names and health details circulate in e mails and shared sheets	Access control and masking of patient data

The repository is likely to justify its modest setup and maintenance cost: it can be a free or low-cost tracking tool that the team already knows how to set up, filling in the fields adds only a few minutes per defect, and in return the hospital obtains the data it needs to decide where to invest, something that was not possible before.

Task 10 – Evaluate Defect Examples and Prevention Strategies 4 Marks

CO1 | Bloom's Level: K5 – Evaluate

Choose three incidents that belong to different defect classes.

For each chosen defect, which single process change would have prevented it?

Discuss:

- The root cause of each chosen defect
- A prevention strategy for each one (for example, requirement reviews, design inspection, coding standards, test oracles)
- The cost and effort of each strategy weighed against its benefit
- Which strategy offers the hospital the best value

Your recommendation must be backed by evidence from the case and the defect data.

Evidence Expected

A root-cause analysis of three defects with justified prevention strategies that take cost into account.

Ans:

Root Causes and Prevention Strategies for Three Defects

REQUIREMENTS	DESIGN	CODING	TESTING
Business-rule review	Concurrency inspection	Validation standards	Oracle review
Prevents I2 / I3 / I5	Prevents I1 / I6	Prevents I4	Prevents I8
Early low-cost check	Architecture check	Reusable code check	Release confidence

Figure 10. Prevention barriers across the life cycle

Three incidents from different defect classes were selected: I3 (requirements), I6 (design) and I5 (coding). For each, I identify the root cause, name one process change that would have prevented it, and weigh the cost of that change against its benefit.

Table 20: Root cause and prevention strategy for the chosen defects

Incident (class)	Root cause	Prevention strategy (one process change)
I3 – incorrect fee charged (requirements)	The fee and tax rule was never documented or agreed with the billing department; it was implemented twice (display and payment) from different interpretations	A requirement review with billing staff and a written fee-rule table; the fee is calculated in one place only and tests use the table as the oracle
I6 – wrong patient on the prescription (design)	The design shares data across users and no one examined it for simultaneous use; no design review took place	A design inspection with the checklist item "shared data and simultaneous users", plus a concurrency test for saving prescriptions
I5 – no senior priority on the 60th birthday (coding)	The age rule was coded without a boundary check, and no one else read the code or tested 59/60/61	A coding standard for date and age calculations, a code review, and unit tests using boundary values 59, 60 and 61 for every rule

For the cost comparison I assume a person-hour costs ₹800 and that a defect escaping its own phase costs roughly ₹32,000 on average to fix later (the mean of the testing and production figures in the case: (1,60,000 + 24,00,000) ÷ 80 defects). The number of additional defects caught by each strategy is taken from the Task 3 model (reviews at 60% efficiency) and, for coding, from lifting coding-phase detection from 17.6% to 35% of the 85 defects present.

Table 21: Cost and benefit of each strategy for a single release

Strategy	Effort (person hours)	Cost (₹)	Additional defects caught early	Saving for each defect (₹)	Total saving (₹)	Benefit ÷ cost
Requirement review (covering all requirements)	50	40,000	16 (18 caught rather than 2)	31,500 (32,000 − 500)	5,04,000	12.6
Design inspection	75	60,000	17.4 (20.4 rather than 3)	30,500 (32,000 − 1,500)	5,30,700	8.8
Coding standard + code review + boundary-value unit tests	80	64,000	14.8	29,000 (32,000 − 3,000)	4,27,750	6.7

All three strategies are worthwhile, since each returns more than six times what it costs. The savings overlap (a defect can be caught only once), so they cannot just be summed, but the ranking by value is clear.

Best value: the requirement review. It has the best benefit-to-cost ratio (around 12.6), needs only a few hours of discussion with billing and OPD staff, requires no tools and targets the largest group of defects (30 of 100). It would have prevented I3, I2 and I10, and it underpins the later strategies, because the test oracles are drawn from the reviewed requirements. The design inspection has a lower ratio (around 8.8), yet I would still keep it, because it deals with the Critical defect I6, where the cost of a single occurrence (a privacy breach) far exceeds the ₹40,000 assumed in the model.

Task 11 – Recommend and Defend a Quality Improvement Plan 4 Marks

CO1 | Bloom's Level: K5 – Evaluate

Drawing on your analysis in Tasks 1–10, propose a quality improvement plan for MediQueue.

Discuss:

- The process changes you propose, in order of priority
- How testers, reviews and the defect repository fit into your plan
- The expected impact on the number and cost of defects
- The trade-offs, risks and limitations of your plan
- Whether the plan is still workable for a small team of five developers

Your recommendation must arise from your own analysis. A generic list of best practices will not be enough.

Evidence Expected

A technically justified quality improvement plan backed by the evidence built up in earlier tasks.

Ans:

Quality Improvement Plan for MediQueue

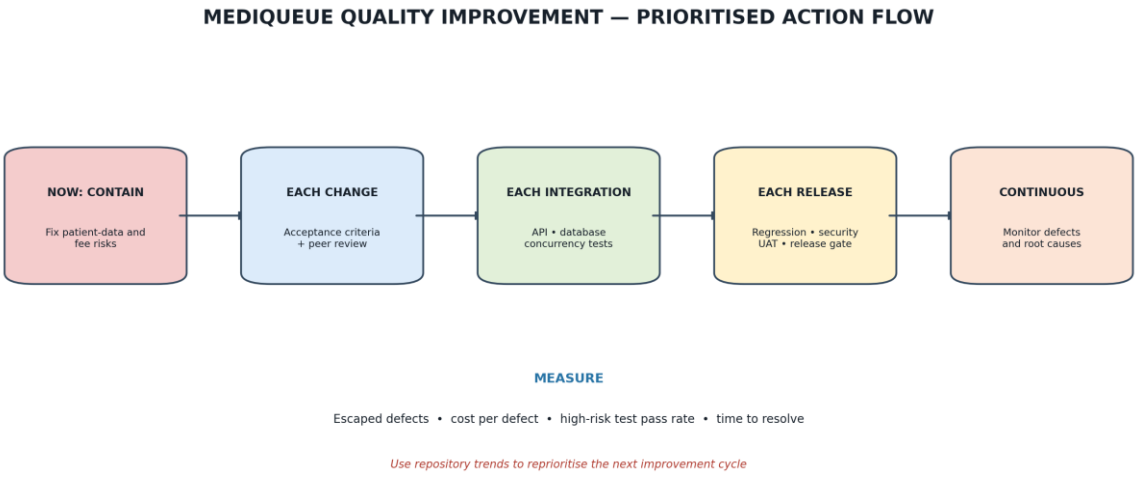


Figure 16. Prioritised quality improvement roadmap

Figure 11. Quality improvement roadmap

The proposed plan is based on the evidence from Tasks 1–10: half the incidents (and 52% of the defects) begin in requirements and design; only 9.6% of these are caught early; production defects account for 91.9% of the cost; and there is no independent tester, no exit criterion and no repository. The plan is ranked by value for money and by what a five-developer team can realistically achieve.

Table 22: Quality improvement plan in priority order

Priority	Change	Role of the tester / reviews / repository	Addresses	Effort
1	Release gate: documented exit criteria (no open critical or high defect, planned high-risk tests completed, at least 95% passed) signed off by the hospital	The tester assembles the evidence; management makes the decision	Approval based on a demo; all incidents	A few hours per release
2	Review of every rule (fees, working days, age rules, performance target) with billing and OPD staff; maintain a business-rule table	Tester and developers carry out the review; the table serves as the test oracle	I2, I3, I5, I10	About 50 person-hours

3	Appoint an independent tester (full-time, or a hospital IT staff member trained for the role)	Prepares the test plan, designs tests using equivalence classes and boundary values, executes system tests and reviews test cases (I8)	I4, I5, I7, I8, I10	Roughly 8 months of one person's time
4	Design inspection of high-risk areas: shared data, concurrency and external interfaces; add a concurrency test and a load test	The tester and two developers; checklist held in the repository	I1, I6, I9	About 75 person-hours
5	A defect repository with the Task 9 fields; a monthly review of defects by class, phase and cost	Entire team; the tester owns the reports	Learning from every incident	1 day to set up, 5 minutes per defect
6	Coding standard, peer review and boundary-value unit tests by the developers	Developers	I4, I5	About 80 person-hours

Priority	**Change**	**Role of the tester / reviews / repository**	**Addresses**	**Effort**
7	Document review: walk through the manual on the working application; interface monitoring and alerting for the SMS provider	Tester; operations	I7, I9	A few hours

Expected impact on the number and cost of defects

I applied the Task 3 defect-flow model with these assumptions: requirements and design reviews remove 60% of defects present, coding-phase detection rises from 17.6% to 35% (code review and unit tests), and testing-phase detection rises from 26.7% to 60% (planned tests and an independent tester).

Table 23: Expected outcome of the plan for a single release (of the same size as the last one)

Measure	**Last release**	**With the plan**
Defects detected before release	40	79.6
Defects reaching patients	60	20.4
Cost of fixing the defects (₹)	26,10,500	10,95,412
Cost of the plan (tester ₹4,00,000; reviews ₹1,00,000; code review and unit tests ₹64,000; training and tools ₹50,000) (₹)	–	6,14,000
Total cost (₹)	26,10,500	17,09,412

Net saving (₹)	–	9,01,088 (34.5%)

The plan reduces the defects reaching patients by roughly two thirds (from 60 to about 20) and saves around ₹9 lakh per release after covering its own cost, not counting benefits that are not quantified, such as fewer patient-safety and privacy incidents.

Trade-offs, risks and limitations

- Slower first release: reviews and testing consume calendar time; the hospital has to accept a gate that may delay a release.
- Assumed efficiencies: The 60%, 35% and 60% figures are estimates; if reviews are poor, the saving will be smaller. I would track them in the repository and update them. ● Review as ritual: reviews could turn into a box-ticking exercise; checklists and recorded defects are needed to keep them meaningful.
- Friction between tester and developers: the tester has to be seen as part of the team; defects are reported without blame.
- Cost not counted: patient harm and loss of trust are not included in the figures, so the actual benefit is greater.
- **Single release data:** the estimate is based on a single release with 100 defects.

Is it workable for five developers?

Yes, provided it is kept lightweight. The additional developer effort is about 125 person-hours for the two reviews plus 80 hours for code review and unit tests, roughly 205 hours in all. Five developers over eight months provide 5 × 8 months × 20 days × 8 hours = 6,400 hours, so the plan uses about 3.2% of developer time, plus one tester. Reviews are short (90
minutes), checklist-driven and focused on the highest-risk parts, and the repository tool can be free. The only major cost is the tester, who is also the biggest source of savings.

DAY-4 DESIGN CHALLENGE

Task 12 – Re-evaluate and Adapt the Quality Process 8 Marks

CO1 | Bloom's Level: K5 – Evaluate

Changed Requirement

The hospital now wishes to release a new MediQueue version every two weeks rather than every six months.

In addition, MediQueue has to connect to the national digital health ID system, which an external agency maintains.

Patient health records will from now on be shared across hospitals.

Reassess your original solution in light of these changed conditions.

You may:

RETAIN → MODIFY → or REDESIGN

your original solution.

A. Impact Analysis 2 Marks

CO1 | Bloom's Level: K5 – Evaluate

Discuss:

- What is different from the original situation
- Which assumptions in your quality improvement plan are impacted
- Why frequent releases alter the role of testing and of the tester
- Why sharing health records increases the severity of certain defect classes

Evidence Expected

A clear analysis of how the new conditions impact the original quality improvement plan. **Ans:**

Impact Analysis of the New Conditions

What has changed

Table 24: Original versus changed situation

BEFORE: ORIGINAL CONTEXT	AFTER: CHANGED CONTEXT
Release every six to eight months	Release every two weeks
Mostly hospital-controlled application	National digital health ID agency controls an external interface
One-hospital record workflows	Health records shared across hospitals
Main focus: internal functional quality	Additional focus: identity, consent, privacy, interoperability and partner availability
Long manual test window	Continuous automated checks and risk-based release gates
↓	↓
PERIODIC TESTING	**CONTINUOUS, INTEGRATION-FOCUSED QUALITY**

Figure 15. MediQueue quality context: before and after the change

Aspect	Original situation	Changed situation
Control	The team controls all its own code and releases	The agency controls when and how the ID system changes

Assumptions in my plan that are impacted

- One large review per release: The plan assumes whole-document reviews (around 125 person-hours). In a two-week cycle this effort cannot be repeated every release; reviews have to be small and tied to each change.
- Manual testing by a single tester: A complete manual regression run (say 400 cases × 6 minutes = 40 person-hours) would consume half of the 80 hours one tester has in a two-week cycle.

- A test bed we control: With an external ID system, the test environment is the agency's sandbox, which may not behave the same as the live system.
- A repository for a single system: The repository assumes every defect lies in our code; some will now lie in the agency's system or in the contract between the two. ● A cost of ₹40,000 per production defect: This increases when a defect can

propagate incorrect data to other hospitals and each release is live for only two weeks.

Why frequent releases alter the role of testing and of the tester

With a release every two weeks, there is no "testing phase at the end". Testing has to run continuously alongside development: a tester who only joins at the end would be a constant bottleneck. The tester joins the team from story refinement onwards, spends less time on repetitive checks (which are automated and run on every change) and more on risk analysis, exploratory testing, test automation and the release decision. Every release must also be reversible and monitored, because a faulty release will be live for 6,000 patients a day.

Why sharing health records increases the severity of certain defect classes Once records are shared, an I6-type defect (another patient's data in a document) stops being a local error and becomes a confidentiality breach across hospitals, carrying legal obligations and possible harm to a patient treated on incorrect information. Design and data defects (shared state, identity matching) and interface defects therefore become Critical, as would an access-control defect previously rated Medium. Requirement defects concerning consent and identity also become Critical. Defects like I7 (manual) stay Low.

B. Evaluation of the Original Plan 2 Marks

CO1 | Bloom's Level: K5 – Evaluate

Assess whether the plan you recommended is still suitable.

Discuss:

- Whether the planned reviews and test activities can fit within a two-week cycle ● Which new defect origins the external health ID system brings in
- Whether your defect repository is still adequate

Evidence Expected

A technically supported assessment of how suitable the original plan is under the new conditions.

Ans:

Is the Original Plan Still Suitable?

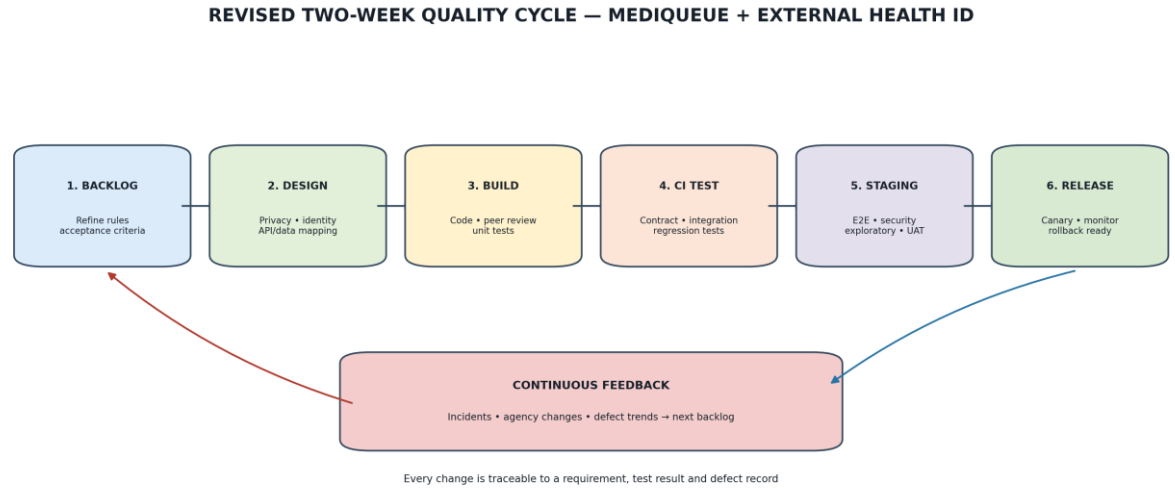


Figure 17. Revised two-week quality cycle for MediQueue

Figure 12. Revised two-week delivery pipeline

Table 25: How well the planned activities fit a two-week cycle

SPRINT STAGE	WHAT THE TEAM DOES	QUALITY OUTPUT
Days 1–2: Refine	Review only the rules and acceptance criteria in the sprint stories (about 30–45 minutes per story); maintain the shared rule table.	Approved story criteria

Days 2–3: Design	Review changes affecting privacy, identity, API contracts, data mapping or concurrency.	Reviewed design and risks
Days 3–7: Build	Code, peer review, run unit tests and static checks on each change.	Reviewed code and unit results
Days 5–9: Integrate	Run API contract, integration, security and automated regression tests.	Integration and regression evidence
Days 9–10: Validate	Run exploratory, usability and privacy checks; confirm critical user journeys in staging.	Acceptance evidence
Days 11–12: Release	Apply release criteria, staged rollout, monitoring and rollback readiness.	Release decision and rollback plan
CONTINUOUS	**Monitor external service changes and production incidents; feed learning into the next backlog.**	**Updated tests and risk register**

Figure 16. Practical two-week sprint quality schedule

The capacity numbers are my own estimates: the original plan's effort (roughly 1,485 person hours per six-month release) divided by 13 for the smaller change in each release, plus one full regression run and the new checks.

New defect origins brought in by the external health ID system

- Contract and version defects: the agency alters a field, version or rule without informing us (the same pattern as I9).
- Data-mapping and identity defects: patients matched incorrectly, inconsistent name and date formats, duplicate IDs.
- Availability and speed: the external service is slow or unavailable; our application freezes or loses the booking.
- Authentication and consent defects: expired tokens, missing patient consent, incorrect access rights.
- Environment differences: the agency's test environment behaves differently from the live service.
- **Privacy and security defects:** records exposed in transit or in logs.

Is the defect repository still adequate?

No. It was built for defects in the team's own system. It requires extra fields: whether the defect is internal, in the agency's system or in the interface; the interface version or contract reference; the agency's ticket number and response date; data sensitivity and the number of patients or hospitals affected; whether the defect must be reported to the authority; the release and pipeline stage where it was detected; and the time taken to detect it and restore service. Without these, the team cannot distinguish its own defects from the agency's or see how quickly a problem was detected and fixed.

C. Adaptation or Redesign 2 Marks

CO1 | Bloom's Level: K5 – Evaluate

Decide whether to:

Retain, modify or redesign the quality process.

If you modify or redesign it:

- Describe the revised process and the revised role of the tester
- Explain which testing principles gain greater importance now
- Explain how defects coming from the external system will be classified and tracked If you keep the original plan, give adequate justification for why no change is needed.

Evidence Expected

A suitable response to the changed requirement with clear quality-engineering reasoning. **Ans:**

Decision: Modify the Quality Process

The recommended decision is to MODIFY the plan. I do not retain it unchanged, because Table 25 shows its reviews and manual testing cannot fit a two-week cycle (roughly 31% over capacity), and because it offers no way to handle an external system. I do not redesign it from scratch, because the evidence it rests on is still sound: over half the defects begin in requirements and design, early detection is far cheaper (₹500 or ₹1,500 compared with ₹40,000), and an independent tester, a release gate and a repository are still required. What changes is how frequent and how small each activity is, and what gets automated.

The revised process

Figure 2: Revised quality cycle for a two-week release

- Story refinement: every story receives acceptance criteria and a business-rule check by the tester and the hospital owner of that rule (30–45 minutes).
- Design check: only for high-risk stories (shared data, identity, external interface), using the checklist.
- Development: peer review and boundary-value unit tests form part of the definition of done.
- Automated pipeline: every change triggers automated regression tests on the critical paths (booking, token, fee, prescription, reminder) and contract tests against the health ID sandbox. The manual regression run of about 40 hours is replaced by an automated run lasting a few minutes.
- Tester checks: exploratory testing of the changed stories, concurrency and load tests in the nightly run, and privacy checks for any change affecting health records. ● Release gate and safe rollout: exit criteria are verified from the pipeline results; the release goes first to a small group and can be rolled back; interfaces (SMS, health ID) are monitored with alerts.
- **Retrospective:** a defect repository review in every cycle to tune checklists and tests.

The tester's revised role

The tester becomes a quality coach and test engineer embedded in the team: analysing the risk of each story, designing and maintaining the automated regression and contract tests, carrying out exploratory and privacy testing, owning the release-gate evidence and repository reports, and serving as the contact point with the agency for interface problems. The tester is no longer just a final checker.

Principles that gain importance

● Early testing and planning: in two weeks there is no time for late correction. ● Repeatable, reusable tests (regression): every release has to re-prove existing behaviour; automation is essential.

- Testing depends on context: health data and money demand stricter testing than the manual does.
- Defect clustering: the interface and shared-data areas are where most new defects will be found.
- Test using invalid inputs and at the interfaces: incorrect, delayed or missing responses from the agency are normal cases that must be tested.

Classifying and tracking defects from the external system

Table 26: Additional classification for external-system defects

Item	How it is managed
Class	A new "External interface" class with sub-classes: contract/version, data mapping, availability/performance, authentication/consent and environment difference
Owner	Internal (our adapter), agency (their service) or joint (contract unclear); determines who fixes it
Severity	Based on impact on patients and data: any incorrect record or privacy exposure is Critical
Tracking	Our defect ID is linked to the agency's ticket; service-level times are logged; any workaround is recorded (queue, retry, manual process)
Measures	Time to detect, time to restore, number of such defects per release and proportion caused by agency changes

Item	How it is managed
Prevention	Contract tests on each build; production monitoring; a named contact and a shared release calendar with the agency

D. Evidence-Based Defence 2 Marks

CO1 | Bloom's Level: K5 – Evaluate

Justify your final decision with evidence.

Use:

- The defect data and the cost figures
- Defect origins and classes
- Release frequency
- Risks to patient safety and data privacy
- Team size and capacity

Your final decision must follow logically from the analysis you have carried out.

Evidence Expected

A well-supported final decision that brings together defect evidence, process reasoning and the changed constraints.

Ans:

Defending the Final Decision

Table 27: Evidence in support of the decision to modify the plan

Evidence	What it shows	Impact on my decision
Defect data and cost	Patients found 60% of the defects, accounting for 91.9% of the total cost (₹24,00,000 out of ₹26,10,500); a production defect costs 80 times as much as a requirement defect	The early-detection principle remains, so I do not discard the plan
Origins and classes	50% of incidents and 52% of defects arise in requirements and design; coding accounts for only 20% of incidents	Retain requirement and design checks, but in a small, per-story form
Release frequency	26 releases per year; the original plan requires about 184 person-hours per cycle against roughly 140 available (31% over)	Keeping the plan unchanged would fail; automation and smaller reviews are required
Patient safety and privacy	Shared records turn I6-type defects into breaches across hospitals (Critical)	Add a design check, privacy testing and a strict release gate
External agency	A new defect origin beyond our control (contract, availability, mapping)	Add contract tests, monitoring and a new repository class
Team size	Five developers (roughly 6,400 hours over eight months) plus one tester	The changes consume about 3% of developer time plus the tester; a complete redesign with a bigger test team would be unaffordable

Reasoning: if I kept the plan unchanged, the manual review and test effort would exceed the team's capacity by 31%, and the team would either drop activities (returning to the past, when 60 defects escaped) or delay every release. If I redesigned everything, I would discard
the parts that the case data strongly support, at a cost a five-person team cannot bear. The modified plan retains the proven elements, scales them down to fit two weeks and automates the repeated checks.

Estimated benefit: with the effect assumed in Task 11 (around 20 defects reaching patients rather than 60 per release-sized amount of work), the expected saving was roughly ₹9 lakh per old-sized release. The same quality improvements are achievable at the new pace, and the release gate, monitoring and rollback restrict the damage from any defect that still escapes to two weeks of exposure instead of six months.

Risks I accept and how I would monitor them: the automated regression suite takes time to build (early cycles will have lower coverage, so the tester should begin with the critical paths); the review and test efficiencies are estimates (the repository will reveal the real figures after three cycles); and the agency's behaviour lies outside the team's control (contract tests and monitoring provide early warning). If the repository shows that agency interface defects dominate and cannot be controlled by the team, I would reconsider the decision and look at redesigning the process around a dedicated integration test group and a formal agreement with the agency.

MARK DISTRIBUTION

Task	Assessed area	Marks
Task 1	Evaluate Testing as an Engineering Activity	5
Task 2	Evaluate the Role of Process in Software Quality	2
Task 3	Evaluate Testing as a Process	6
Task 4	Apply and Critique Basic Definitions	4
Task 5	Evaluate the Application of Testing Principles	3
Task 6	Evaluate the Tester's Role	3
Task 7	Trace the Origins of Defects	4
Task 8	Evaluate Defect Classes and Severity	4
Task 9	Design and Appraise a Defect Repository	3
Task 10	Evaluate Defect Examples and Prevention Strategies	4
Task 11	Recommend and Defend a Quality Improvement Plan	4
Task 12	Day-4 Design Challenge (A–D, 2 marks each)	8
Total		50